SCENARIO-BASED CONCURRENCY INTERVIEW QUESTIONS

1. Race Condition In Shared Counter - Lost Updates

Scenario:

Multiple checkout requests hit the WalletService at the same time. Each request reads the user's balance, deducts the amount, and writes the new balance back. Occasionally the final balance is wrong (money disappears or is overspent).

Thought Process:

They want to see if you understand race conditions and why 'read-modify-write' is unsafe without synchronization.

Solution Design:

Make the balance update atomic. In Java, avoid naive balance = balance - amount logic across threads. Options:

• Use synchronized / ReentrantLock to serialize updates per user.

• Use database-level atomic UPDATE ... SET balance = balance - ? WHERE user_id = ? AND balance >= ?.

• Use AtomicLong/AtomicInteger for in-memory counters that must be thread-safe.

```
private final ReentrantLock lock = new ReentrantLock();
public void debit(String userId, long amount) {
lock.lock();
try {
long bal = repo.getBalance(userId);
if (bal < amount) throw new InsufficientFunds();
repo.updateBalance(userId, bal - amount); // single writer section
} finally {
lock.unlock();
}
}
```

Interview Takeaway:

Any read-modify-write across threads must be guarded or made atomic. Never trust plain in-memory fields for shared mutable state.

2. Deadlock Between Services / Threads - System Freezes Under Load

Scenario:

OrderService locks orderId 123 then calls PaymentService, which locks paymentId P-789 and calls back into OrderService to mark status. Under heavy load, threads wait forever and CPU usage is low but requests never finish.

Thought Process:

They're checking deadlock reasoning: circular lock acquisition (A waits for B while B waits for A).

Solution Design:

Fix by enforcing a global lock ordering rule. Example: always lock payment first, then order. In Java code, avoid nested synchronized blocks on different monitors in arbitrary order.

Also add timeouts on locks so you fail fast instead of hanging forever.

```
boolean acquiredA = lockA.tryLock(50, TimeUnit.MILLISECONDS);
boolean acquiredB = lockB.tryLock(50, TimeUnit.MILLISECONDS);
if(!(acquiredA && acquiredB)) {
// rollback / retry instead of deadlocking forever
}
```

Interview Takeaway:

Deadlocks are usually design issues, not 'GC problems'. Show that you understand lock ordering and timed locking.

3. Unbounded Thread Creation - CPU Spikes and OutOfMemoryError

Scenario:

A REST endpoint starts a new Thread for every request to generate reports. Traffic grows and suddenly the JVM dies with OutOfMemoryError: unable to create new native thread.

Thought Process:

They're testing if you know why creating raw threads per request is dangerous and how to size thread pools.

Solution Design:

Use a bounded ExecutorService instead of new Thread(). With Spring Boot you expose this via @Async and a custom TaskExecutor.

You cap max threads and use a queue to smooth bursts.

```
@Bean
public Executor taskExecutor() {
ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
exec.setCorePoolSize(10);
exec.setMaxPoolSize(20);
exec.setQueueCapacity(100);
exec.setThreadNamePrefix("report-");
exec.initialize();
return exec;
}
@Async
public void generateReport(...) {
// heavy work here
}
```

Interview Takeaway:

Concurrency must be controlled. Bounded pools protect the JVM and the machine. 'MaxPoolSize' and 'QueueCapacity' are levers.

4. Feature Flag Not Turning Off - Threads Still See Old Value

Scenario:

You added a boolean killSwitch = false; in a singleton. You flip it to true at runtime to stop a risky experiment, but some request handlers keep acting like it's still false.

Thought Process:

They're probing if you understand memory visibility, CPU caches, and 'happens-before' rules in Java.

Solution Design:

Mark shared state as volatile or use AtomicBoolean so changes are visible across threads immediately.

Without volatile, one thread may read a cached value forever.

```
private static final AtomicBoolean killSwitch = new AtomicBoolean(false);
public void disableFeature() {
killSwitch.set(true);
}
public void handleRequest() {
if (killSwitch.get()) {
throw new FeatureDisabledException();
}
// continue normal flow
}
```

Interview Takeaway:

Thread safety is not just about 'no two threads write at once'. Visibility matters. Use volatile / Atomic* for shared flags.

5. Checkout API Feels Slow - Thread Is Busy Doing Extra Work

Scenario:

The /checkout endpoint processes payment, generates invoice PDF, sends email, writes audit logs, and only then returns 200 OK. P95 latency is terrible.

Thought Process:

They want to see if you can separate 'user-facing work' from 'background work' and keep HTTP threads free.

Solution Design:

Return early after essential steps (charge + order ID). Offload slow extras to async tasks / message queue.

In Spring Boot: controller thread just enqueues work to an Executor or Kafka topic and returns immediately.

```
@PostMapping("/checkout")
public ResponseEntity<OrderResponse> checkout(...) {
OrderResponse resp = coreOrderFlow(); // payment + reserve stock
asyncTasks.publishInvoiceJob(resp);
asyncTasks.auditOrder(resp);
return ResponseEntity.accepted().body(resp);
}
```

Interview Takeaway:

Fast response != do everything synchronously. Senior engineers design latency budgets per step.

6. Aggregator Service Is Too Slow - Needs Parallel Calls

Scenario:

A ProductDetails API calls PricingService, InventoryService, and ReviewService sequentially. Total time is 900ms (300+300+300). Business wants <400ms.

Thought Process:

They're testing if you know how to run independent I/O calls in parallel and then merge results.

Solution Design:

Use CompletableFuture.supplyAsync(...) to call each downstream service in parallel using a thread pool. Then combine results once all complete.

```
CompletableFuture<Pricing> p = CompletableFuture.supplyAsync(() -> pricingClient.get(id), exec);
CompletableFuture<Stock> s = CompletableFuture.supplyAsync(() -> stockClient.get(id), exec);
CompletableFuture<Reviews> r = CompletableFuture.supplyAsync(() -> reviewClient.get(id), exec);
ProductDetails dto = CompletableFuture.allOf(p,s,r)
.thenApply(v -> combine(p.join(), s.join(), r.join()))
.join();
```

Interview Takeaway:

Parallelizing I/O-bound calls is low-hanging fruit for latency wins. Mention thread pool sizing to avoid starving CPU.

7. High Read Volume, Rare Writes - DB Is Getting Hammered

Scenario:

A configuration map (feature toggles, tax rules, etc.) is read thousands of times per second by all requests, but updated maybe once in 10 minutes. DB becomes a hotspot.

Thought Process:

They're checking if you know how to safely cache shared data in-memory without corrupting it under concurrent access.

Solution Design:

Use an in-memory cache + ReadWriteLock. Many readers can access a stable snapshot concurrently. Writers take the write lock rarely to refresh the data atomically.

This avoids per-request DB hits while staying thread-safe.

```
private final Map<String,String> configCache = new HashMap<>();
private final ReadWriteLock rw = new ReentrantReadWriteLock();
public String getConfig(String key){
rw.readLock().lock();
try { return configCache.get(key); }
finally { rw.readLock().unlock(); }
}
public void refreshAll(Map<String,String> newData){
rw.writeLock().lock();
try {
configCache.clear();
configCache.putAll(newData);
} finally {
rw.writeLock().unlock();
}
}
```

Interview Takeaway:

ReadWriteLock is perfect when reads dominate writes. It scales better than 'synchronized getConfig()'.

8. Cron Job Runs Twice in Cluster - Double Billing / Double Emails

Scenario:

You deployed 4 replicas of InvoiceService on Kubernetes. A @Scheduled job runs nightly to send monthly invoices. Now customers get duplicate emails (one per pod).

Thought Process:

They want to know if you understand concurrency at the cluster level, not just inside one JVM.

Solution Design:

Use leader election / distributed lock so only one instance does the job. Options:

• Use DB row 'cron_lock' with SELECT ... FOR UPDATE and TTL.

• Use Redis-based distributed lock.

• Use Kubernetes CronJob (single runner pod) instead of @Scheduled in every pod.

```
@Scheduled(cron = "0 0 1 * * *")
public void runMonthlyInvoicing(){
if(!lockService.acquireLock("monthly-invoice")) return;
try {
invoiceGenerator.run();
} finally {
lockService.releaseLock("monthly-invoice");
}
}
```

Interview Takeaway:

Senior engineers think beyond one JVM. 'Concurrency' also means coordination across replicas.

9. Rolling Deployment Kills In-Flight Tasks - Partial Processing

Scenario:

You deploy a new version. Kubernetes kills the old pod. That pod was still processing 200 refund requests in a background executor. Some refunds never completed and finance is angry.

Thought Process:

They're testing if you know graceful shutdown for thread pools.

Solution Design:

Implement preStop / shutdown hooks. Stop accepting new tasks, wait for running tasks to finish, then exit.

In Spring Boot, use SmartLifecycle or @PreDestroy to call executor.shutdown() and awaitTermination().

```
@PreDestroy
public void shutdownExecutor(){
exec.shutdown();
exec.awaitTermination(30, TimeUnit.SECONDS);
}
```

Interview Takeaway:

Zero-downtime deploys are not just 'blue/green'. You must drain work safely.

10. Queue Buildup Under Peak Load - System Falls Behind

Scenario:

During Diwali sale, requests enqueue faster than workers can process. The in-memory queue grows without limit, latency explodes, and eventually the service crashes.

Thought Process:

They are checking if you understand backpressure and graceful degradation under extreme load.

Solution Design:

Use bounded queues with rejection policies. When the queue is full, you:

• Reject low-priority work with HTTP 429 (rate limit), OR

• Degrade non-critical features, OR

• Shed traffic via API Gateway rate limiting.

Never let unlimited queues eat all memory.

```
BlockingQueue<Job> q = new ArrayBlockingQueue<>(1000);
ThreadPoolExecutor exec = new ThreadPoolExecutor(
10, 20, 60, TimeUnit.SECONDS,
q,
new ThreadPoolExecutor.AbortPolicy() // fail fast when overloaded
);
```

Interview Takeaway:

Surviving peak load is about controlled rejection, not heroic infinite buffering. Show that you understand graceful failure.